

Git

Control de Versiones

Informática II — UTN FRH

Profesor: Ing. Weber Federico



¿Qué es el control de versiones?

2

¿Les pasó esto alguna vez?



Esto no escala. Necesitamos algo mejor.

Un sistema de control de versiones registra los cambios en tus archivos a lo largo del tiempo.
Podés volver a cualquier versión anterior, ver qué cambió y cuándo.

Git: un poco de historia

3

2005 — Linus Torvalds

Creador del kernel Linux. Necesitaba un sistema de control de versiones rápido y distribuido.

Características principales:

- Distribuido: cada copia es un repo completo
- Rápido: operaciones locales, sin red
- Open source y gratuito
- Integridad: cada cambio tiene un hash único

Hoy lo usan millones de desarrolladores en todo el mundo.



Linus Torvalds

Creador de Linux y Git

¿Para qué sirve Git?

4

Guardar snapshots

Fotografías de tu
proyecto en el tiempo

Volver atrás

Si algo sale mal,
volvés a una versión anterior

Ramas paralelas

Trabajar en features
sin romper lo que anda

Colaborar

Trabajar en equipo
sin pisarse (con GitHub)

En esta clase vemos Git local. GitHub lo vemos más adelante.

Formas de usar Git

5

Terminal (CLI)

La más completa.

Control total.

▲ Lo que usamos en clase

GUIs

GitKraken, SourceTree,

GitHub Desktop.

Interfaz visual.

IDE integrado

VS Code, IntelliJ.

Git dentro del editor.

"Git fue diseñado para entornos Unix, por eso en Windows se instala con Git Bash que provee un entorno compatible"

Antes de Git: la terminal

Git se usa desde la línea de comandos.

Necesitamos conocer algunos comandos básicos antes de empezar con Git.

```
$ pwd
/home/fede/proyecto
$ ls
main.c  archivo2.c  specs.md
$ mkdir nueva-carpeta
$ cd nueva-carpeta
$ clear
```

La terminal es la forma más directa de comunicarte con tu computadora.

Comandos básicos de terminal

7

```
pwd
```

¿Dónde estoy? (print working directory)

```
ls
```

¿Qué hay acá? (list)

```
cd carpeta
```

Ir a una carpeta

```
cd ..
```

Volver atrás

```
mkdir nombre
```

Crear una carpeta nueva

```
clear
```

Limpiar la pantalla

```
touch archivo.c
```

Crear un archivo vacío

```
code .
```

Abrir VS Code en esta carpeta

Comandos básicos de terminal

8

vim

Editor potente. No lo vamos a usar, pero Git lo abre por defecto para algunos mensajes. Si aparece: :q! para salir sin guardar, :wq para guardar y salir.

nano

Editor simple y amigable. Muestra los atajos en pantalla. Es el que vamos a usar en clase.

cat

No es un editor. Muestra el contenido de un archivo en la terminal sin abrirlo.

Vamos a la terminal

Instalación de Git

<https://git-scm.com/>



Linux

```
sudo apt install git
```

Mac

```
brew install git
```

Windows

Descargar de git-scm.com
Incluye Git Bash

Verificar instalación:

```
$ git --version  
git version 2.43.0
```

Una vez que la terminal entiende git, es igual en cualquier SO.

Configuración inicial

10

Lo primero: decirle a Git quiénes somos. Se hace una sola vez.

```
$ git config --global user.name "Tu Nombre"  
$ git config --global user.email "tu@email.com"
```

Git usa esta información para identificar quién hizo cada cambio.
Se guarda en el archivo ~/.gitconfig

Para verificar:

```
$ git config --list  
user.name=Tu Nombre  
user.email=Tu mail
```

Nuestro primer repositorio

11

```
$ mkdir mi-proyecto  
$ cd mi-proyecto  
$ git init  
Initialized empty Git repository in .git/
```

mi-proyecto/

.git/

← No tocar esta carpeta

git init crea una carpeta oculta .git

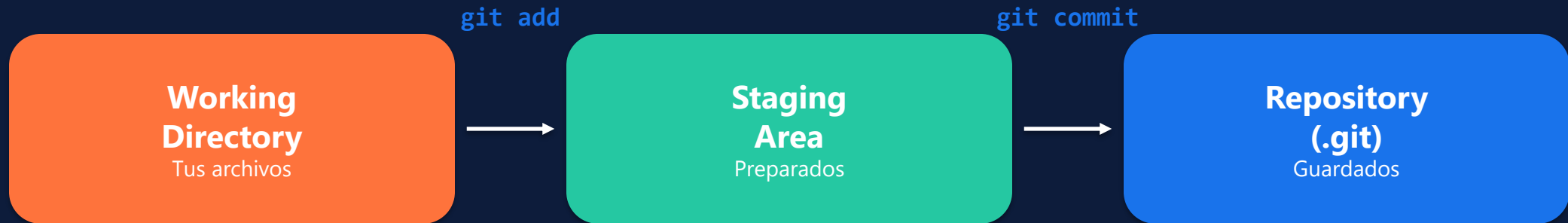
Esa carpeta contiene TODA la información del repositorio.
Mientras exista .git, Git sabe que esto es un repo.

En el prompt aparece la rama actual: (main)

Las tres áreas de Git

12

Este es el flujo fundamental. Entender esto es esencial para entender Git.



Working Directory: donde editás tus archivos

Staging Area: donde preparás lo que vas a guardar

Repository (.git): donde se guardan los snapshots (commits)

Crear un archivo y ver el estado

```
$ touch mimain.c
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include)
        mimain.c
```

tip:

Es buena práctica llamar a la rama principal main. En algunas versiones empieza como master.

```
git branch -m main
```

git status es tu mejor amigo.

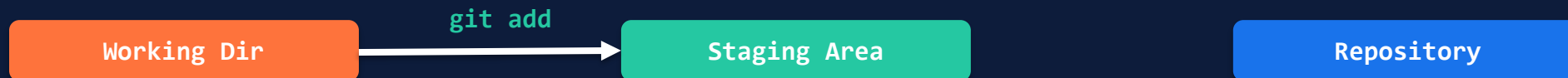
Te dice qué cambió, qué está preparado, y qué falta.

git add: preparar archivos

14

```
$ git add mimain.c
$ git status
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   mimain.c
```

El archivo pasó del Working Directory al Staging Area.



git commit: guardar un snapshot

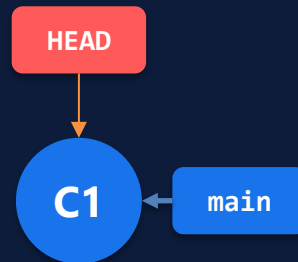
```
$ git commit -m "Primer commit: crear mimain.c"  
1 file changed, 0 insertions(+)
```

Un commit es una "fotografía" del proyecto.

Queda guardado para siempre en el historial.

Si escribís solo git commit (sin -m), se abre vim.

Para salir de vim: ESC → :wq → Enter



HEAD = "donde estás ahora". Es un puntero que marca el commit actual sobre el que estás trabajando.

.gitignore: qué NO rastrear

Modifiquemos el archivo, compilemos y creamos un ejecutable.

```
$ git status
```

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: mimain.c

Untracked files:

(use "git add <file>..." to include in what will be committed)

mimain

no changes added to commit (use "git add" and/or "git commit -a")

¿Qué se ignora?

- Archivos compilados (.exe, .o)
- Carpetas de build
- Configuraciones locales del IDE
- Archivos temporales

.

.gitignore: qué NO rastrear

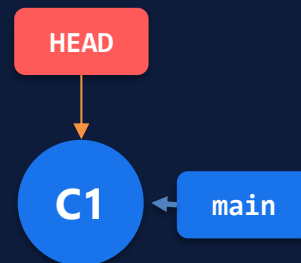
No todo debería ir al repositorio: binarios, temporales, etc. Ver el caso de git add .

Crear el archivo .gitignore:

```
$ touch .gitignore
$ nano .gitignore
#guardar cambios con ctrl+o cerrar con ctrl+x
```

```
# contenido de ejemplo de .gitignore
```

```
mimain
*.exe
*.o
build/
.vscode/
```



```
$ git add .
$ git commit -m "agrego .gitignore y modifico mimain"
```

.gitignore: qué NO rastrear

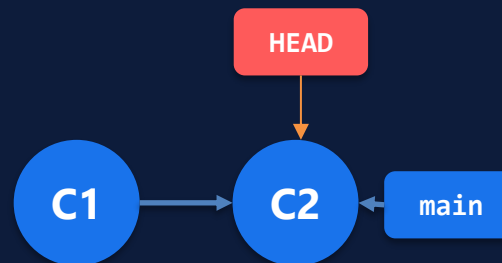
No todo debería ir al repositorio: binarios, temporales, etc. Ver el caso de git add .

Crear el archivo .gitignore:

```
$ touch .gitignore
$ nano .gitignore
#guardar cambios con ctrl+o cerrar con ctrl+x
```

```
# contenido de ejemplo de .gitignore

mimain
*.exe
*.o
build/
.vscode/
```



```
$ git add .
$ git commit -m "agrego .gitignore y modifico mimain"
```

git log: ver el historial

```
$ git log
commit b2c3d4ee7aa456b0860c9950819f6b086cfb4335 (HEAD -> main)
Author: fweber <sweber@frh.utn.edu.ar>
Date: Thu Mar 26 15:48:23 2026 +0000

    agrego .gitignore y modifico mimain.c

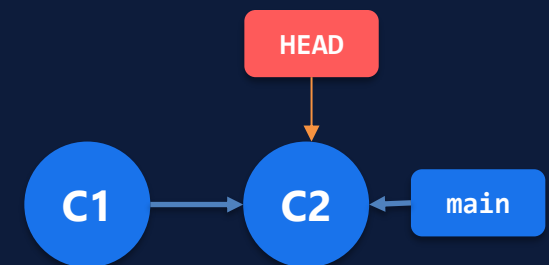
commit a1b2c3d49c46cfd029993d0dc7bbe4126464ed3
Author: fweber <sweber@frh.utn.edu.ar>
Date: Thu Mar 26 15:38:31 2026 +0000

    Primer commit: crear mimain.c
```

Versión resumida:

```
$ git log --oneline
b2c3d4e (HEAD -> main) agrego .gitignore y modifico mimain.c
a1b2c3d Primer commit: crear mimain.c
```

Cada commit tiene un hash único (ej: a1b2c3d). Es como su DNI.

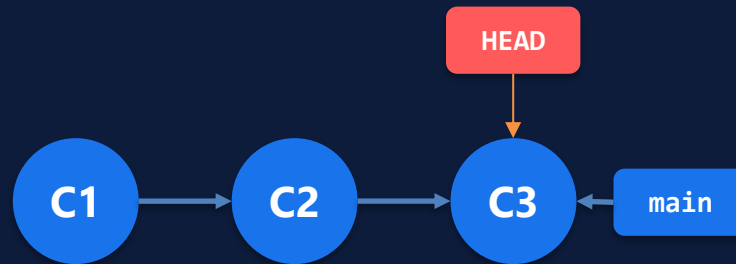


Modificar archivos y commitear

20

Editamos mimain.c, agregamos código, y guardamos otro snapshot:

```
$ git add mimain.c  
$ git commit -m "Agrego función main"  
1 file changed, 5 insertions(+)
```



git diff: ver cambios

```
$ git diff          # cambios no preparados
$ git diff --staged # cambios preparados
```

```
git diff
diff --git a/mimain.c b/mimain.c
--- a/mimain.c
+++ b/mimain.c
@@ -1,3 +1,5 @@
    #include <stdio.h>

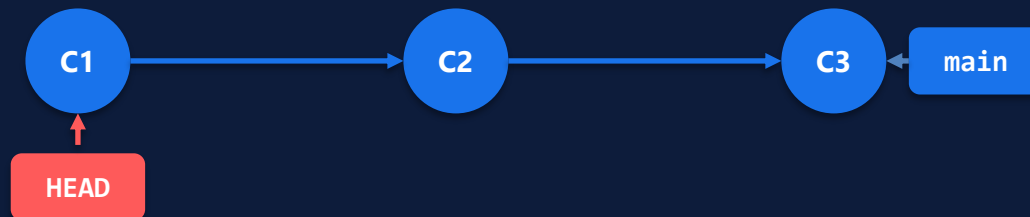
-   int main() {
+   int main(int argc, char *argv[]) {
+       printf("Hola Git!\n");
+       return 0;
    }
```

Las líneas en rojo (-) se eliminaron. Las líneas en verde (+) se agregaron.

Navegar entre commits

```
$ git log --oneline
c3d4e5f Agrego función main
b2c3d4e agrego .gitignore y modifico mimain.c
a1b2c3d Primer commit: crear mimain.c
$ git checkout a1b2c3d
```

Podemos "viajar en el tiempo" a cualquier commit usando su hash.



checkout te mueve a ese punto en el historial. Podés mirar el código, pero estás en "detached HEAD": si hacés commits nuevos acá, los podés perder. Para seguir trabajando, volvé a tu rama con `git checkout main`.

¿Qué es HEAD?

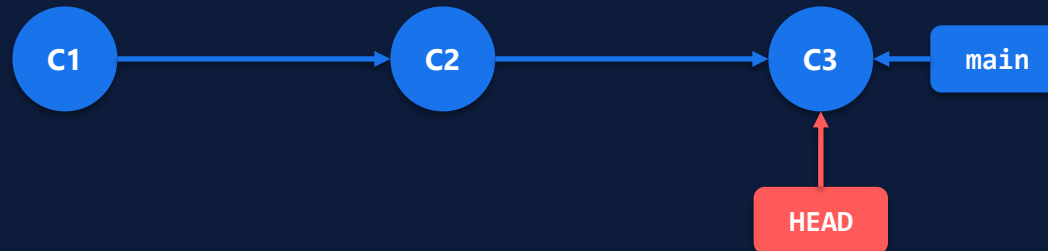
23

HEAD es un puntero que indica "dónde estás ahora".

Normalmente apunta a la rama actual (ej: main).

La rama, a su vez, apunta al último commit.

Analogía: es como el cursor en un documento.



Deshacer cambios: git restore

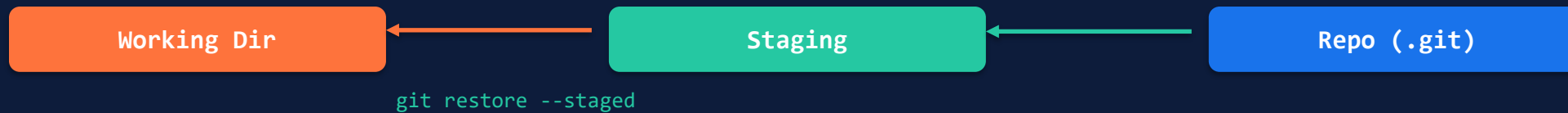
Dos situaciones comunes:

1. Descartar cambios del working directory:

```
git restore archivo.c
```

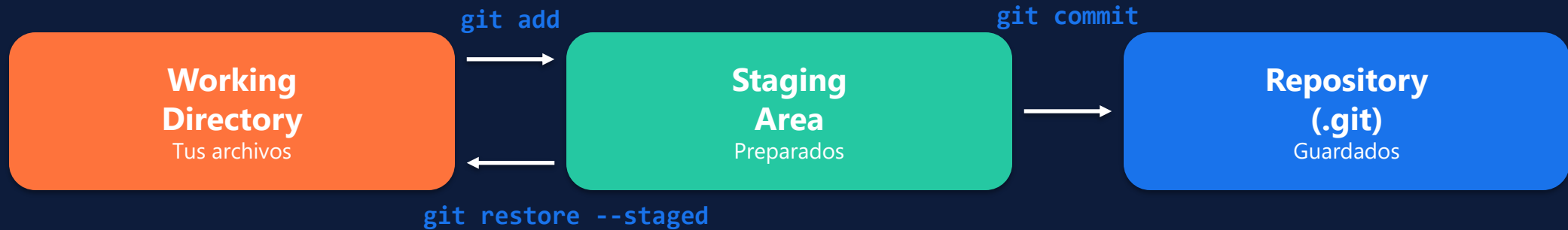
2. Sacar del staging area (des-hacer git add):

```
git restore --staged archivo.c
```



Resumen: flujo básico de Git

25



Comandos esenciales:

- git init** → Crear repositorio
- git status** → Ver estado actual
- git add <archivo>** → Preparar para commit
- git commit -m "..."** → Guardar snapshot
- git log** → Ver historial
- git diff** → Ver cambios
- git restore** → Deshacer cambios

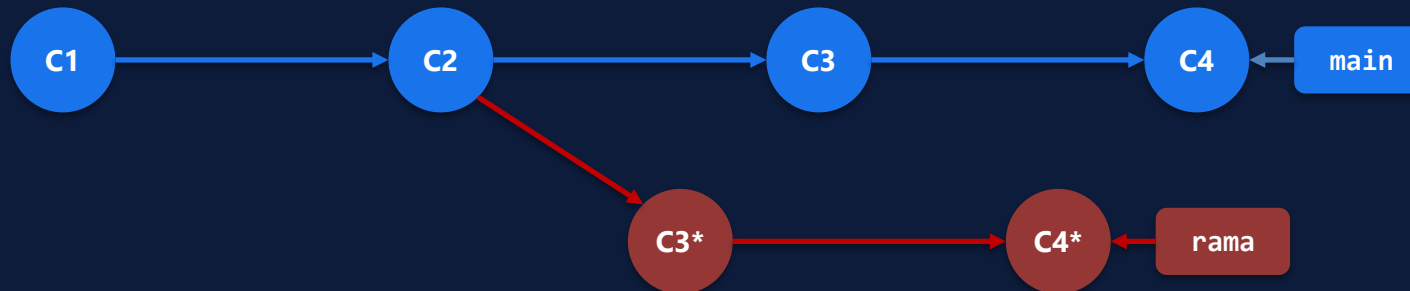
Ramas: la feature más poderosa de Git

Una rama es simplemente un puntero a un commit.

Hasta ahora trabajamos en una sola rama: main.

Las ramas permiten trabajar en paralelo sin afectar el código que ya funciona.

Es como una línea de tiempo alternativa.



Crear y cambiar de rama

27

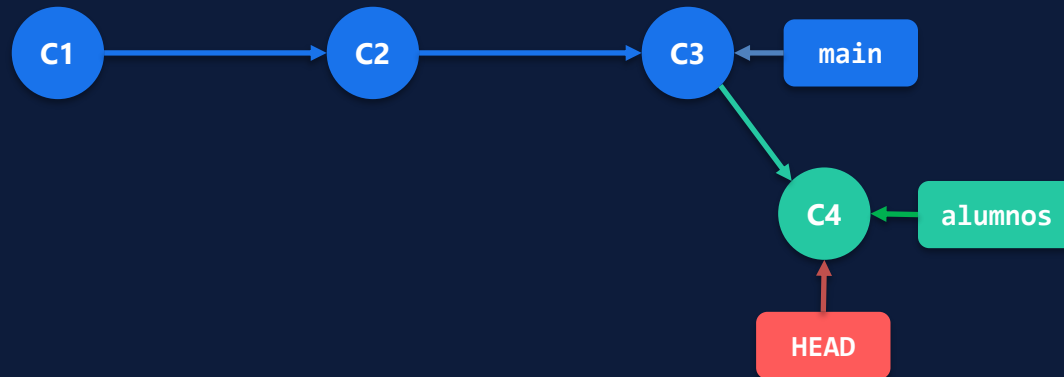
```
$ git branch alumnos
$ git switch alumnos
Switched to branch 'alumnos'
$
# O en un solo paso:
$ git switch -c alumnos
```



Trabajar en una rama

28

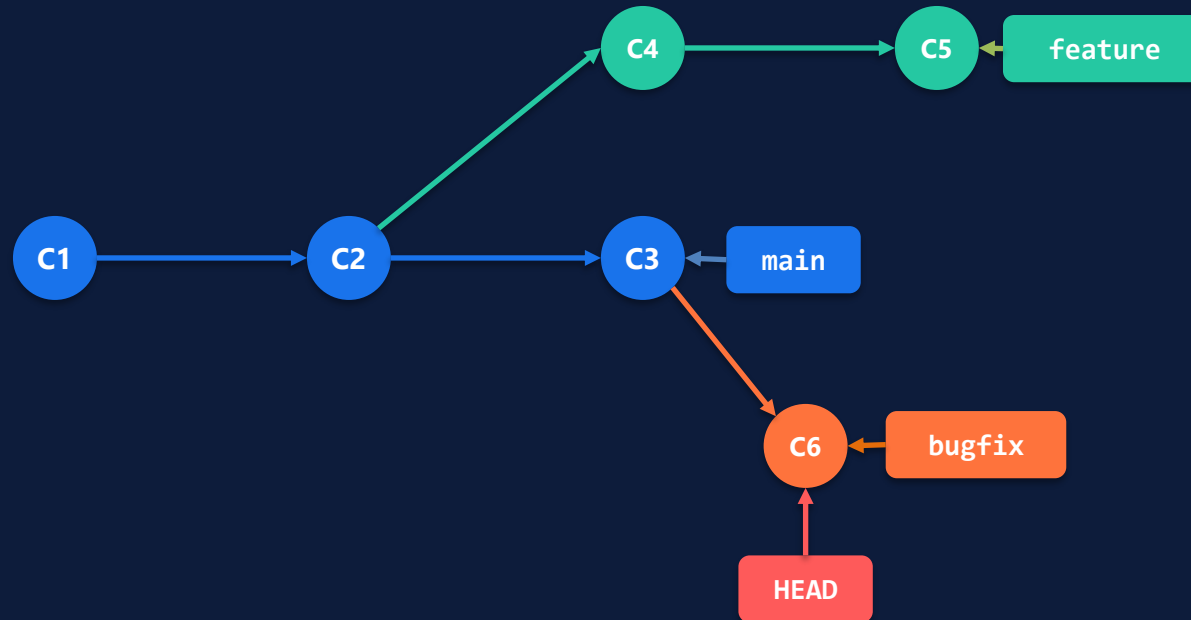
```
$ # Estando en la rama alumnos:  
$ touch alumnos.c  
$ git add alumnos.c  
$ git commit -m "Agrego módulo de alumnos"
```



Ramas paralelas

29

Cada rama es independiente. Los cambios en una no afectan a la otra.



Volver al pasado para trabajar

Antes vimos checkout para mirar un commit viejo. Pero estábamos en "detached HEAD": no podíamos commitear.

¿Y si quiero volver atrás y seguir trabajando desde ahí?

Solución: crear una rama en ese punto del pasado.

```
$ git log --oneline
c3d4e5f Agrego función main
b2c3d4e agrego .gitignore y modifico mimain
a1b2c3d Primer commit: crear mimain.c
# Voy al commit, creo una rama, voy a la rama ...
$ git checkout b2c3d4e
$ git branch fix-viejo
$ git switch fix-viejo
```

Equivalente a:

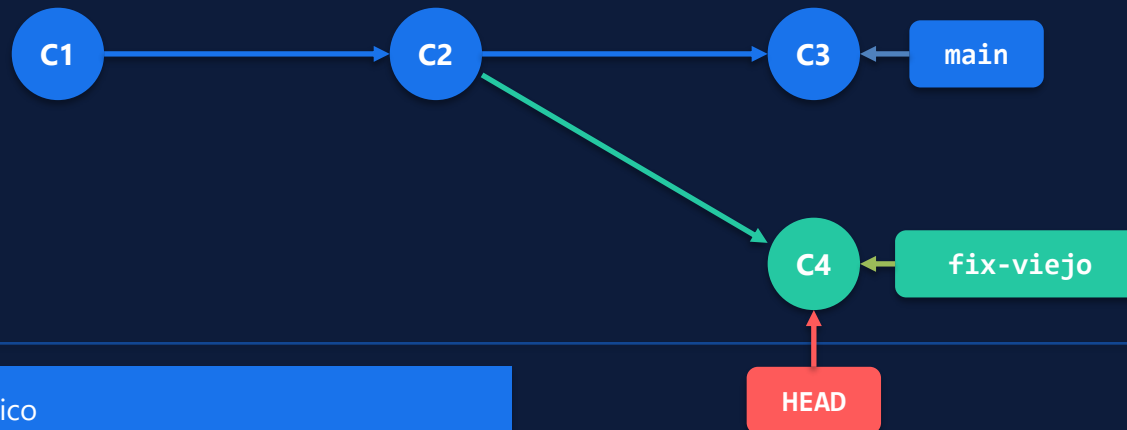
```
$ git checkout b2c3d4e
```

```
$ git switch -c fix-viejo
```

O en un solo paso:

```
$ git switch -c fix-viejo b2c3d4e
```

Ahora puedo commitear en esta rama normalmente.



git merge: unir ramas

Cuando terminamos el trabajo en una rama, la integramos a main:

```
$ git switch main  
Switched to branch 'main'  
$ git merge alumnos
```

¿Qué pasa con el merge?

Depende de si main avanzó o no desde que se creó la rama:

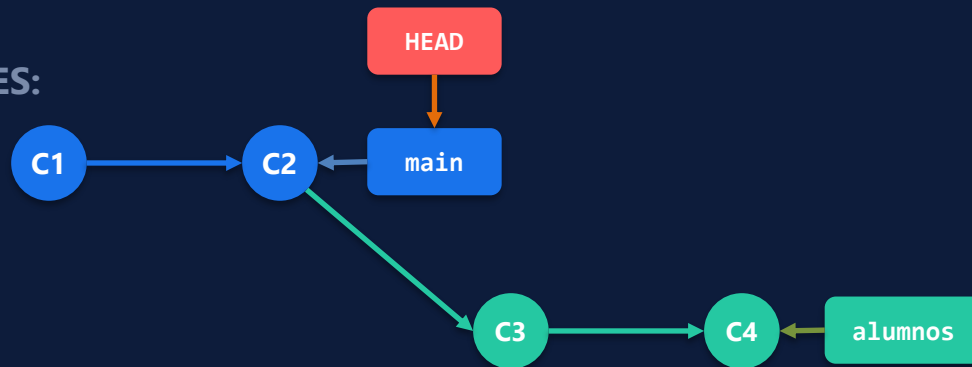
- **Si main NO avanzó:** fast-forward (avance rápido)
- **Si main SÍ avanzó:** merge commit (commit de fusión)

Veamos ambos casos...

Fast-forward merge

32

ANTES:



```
$ git switch main  
Switched to branch 'main'  
$ git merge alumnos
```

DESPUÉS:



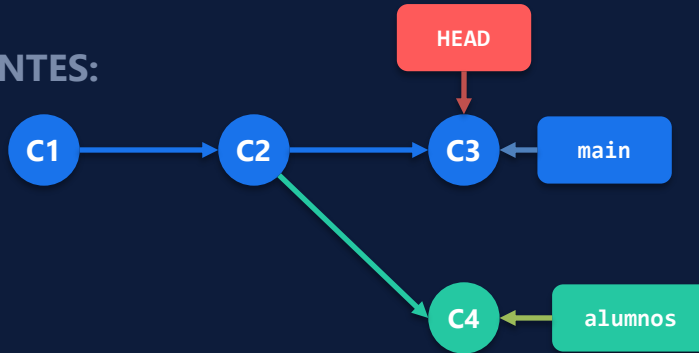
Main no avanzó → Git simplemente mueve el puntero. No se crea un commit nuevo.

Merge commit (3-way merge)

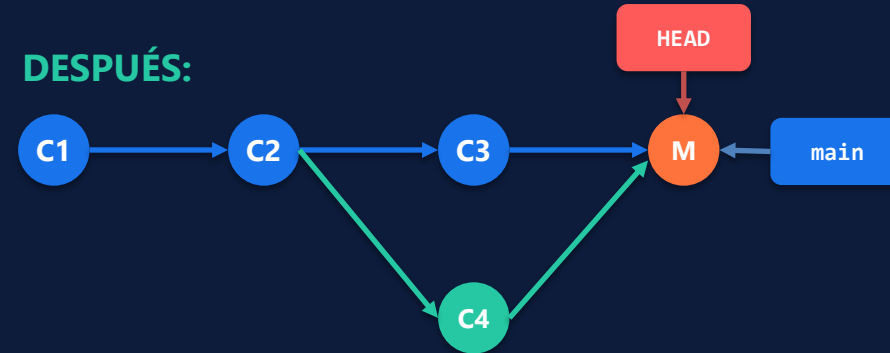
33

```
$ git switch main  
Switched to branch 'main'  
$ git merge alumnos
```

ANTES:



DESPUÉS:



Ambas ramas avanzaron → Git crea un commit de merge (M) que combina los cambios de las dos ramas.

Conflictos de merge

Si ambas ramas modificaron la misma línea, Git no puede decidir solo:

```
<<<<<< HEAD
int x = 10;  // versión de main
=====
int x = 20;  // versión de alumnos
>>>>>> alumnos
```

Pasos para resolver:

1. Editar el archivo: elegir qué código queda
2. Borrar los marcadores (<<<, ==, >>>)
3. `git add archivo.c`
4. `git commit -m "Resuelvo conflicto"`

No es tan común si el equipo se comunica bien.

git stash: guardar cambios temporalmente

Estás a mitad de algo en una rama y necesitas cambiar a otra sin commitear

```
# Estando en la rama alumnos:
$ git switch main
error: Your local changes would be overwritten
Please commit your changes or stash them
$
# Solución: guardar temporalmente
$ git stash
Saved working directory
$ git switch main
Switched to branch 'main'
$
# Cuando vuelvo a mi rama:
$ git switch alumnos
$ git stash pop
# Mis cambios vuelven!
```

Stash es como un cajón: guardás tus cambios y los sacás después.

Eliminar ramas

36

Después de hacer merge, la rama ya no se necesita:

```
$ git branch -d alumnos
Deleted branch alumnos (was d5c3d5a).
$ git branch
* main
```

-d solo funciona si la rama ya fue mergeada.

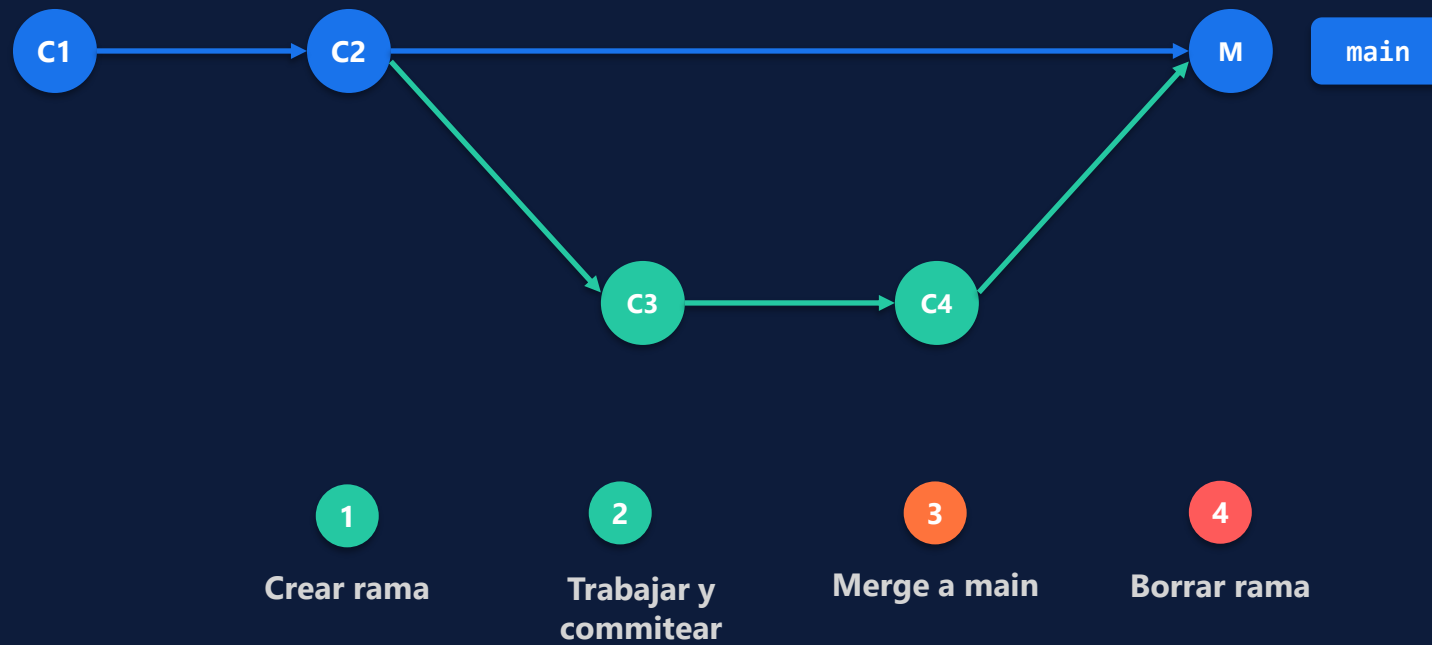
-D fuerza el borrado (cuidado!).

Los commits no se borran.

Solo se borra el puntero. El historial queda intacto.

Flujo completo con ramas

37



Básicos

`git init` Crear repositorio

`git status` Ver estado

`git add <archivo>` Preparar archivo

`git commit -m "..."` Guardar snapshot

`git log --oneline` Ver historial

Cambios

`git diff` Ver diferencias

`git restore <archivo>` Descartar cambios

`git restore --staged` Quitar del staging

Ramas

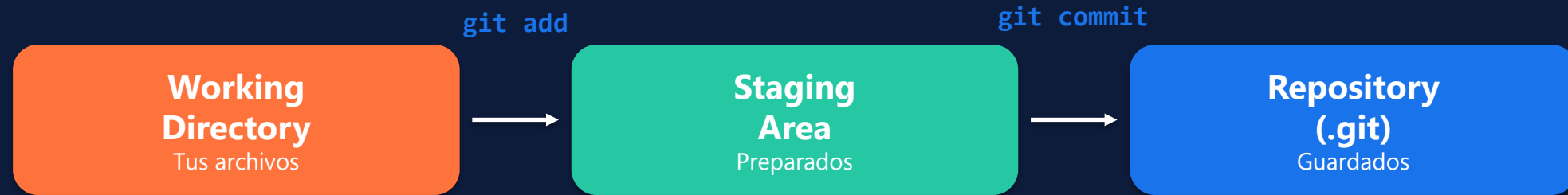
`git branch <nombre>` Crear rama

`git switch <nombre>` Cambiar de rama

`git merge <rama>` Unir ramas

`git stash / pop` Guardar temporal

`git branch -d <rama>` Borrar rama



→ Hacia adelante:

git add (Working → Staging)

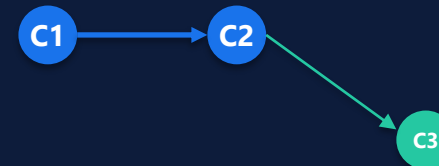
git commit (Staging → Repo)

← Hacia atrás:

git restore (deshacer cambios)

git restore --staged (quitar del staging)

Ramas:



git branch / git switch

git merge / git stash

1. Crear un repositorio nuevo con git init
2. Crear un archivo .c, agregarlo con git add y hacer el primer commit
3. Modificar el archivo, ver los cambios con git diff, y commitear
4. Crear un .gitignore que ignore archivos .exe
5. Crear una rama "feature", hacer cambios y commitear en ella
6. Volver a main y hacer merge de la rama feature
7. Provocar un conflicto de merge y resolverlo manualmente
8. Usar git stash para guardar cambios temporales

¿Preguntas?

Usen Git en todos sus proyectos a partir de ahora.